

Kolekce - pole, zásobník, fronta, slovník

- **kolekce** = datová struktura pro uložení více hodnot
- používá se, když nechceme pracovat jen s jednou proměnnou
- kolekce obvykle obsahuje více prvků stejného typu
- umožňuje:
 - ukládání více hodnot
 - procházení hodnot
 - vyhledávání hodnot
 - přidávání a odebírání hodnot

Příklady kolekcí:

- pole
- list
- zásobník
- fronta
- slovník
- množina

Pole

Co je pole

- **pole** = kolekce prvků stejného typu
- prvky jsou uloženy pod jedním názvem
- k prvkům se přistupuje pomocí indexu
- indexování začíná od 0
- prvky jsou v paměti obvykle uloženy za sebou
- pole má po vytvoření pevnou velikost

Příklad:

```
int[] ciska = { 10, 20, 30 };
```

Indexy:

```
ciska[0] = 10  
ciska[1] = 20  
ciska[2] = 30
```

Výhody pole

- rychlý přístup podle indexu
- jednoduchá struktura
- vhodné pro pevný počet prvků
- efektivní pro procházení cyklem

Nevýhody pole

- pevná velikost po vytvoření
- vkládání doprostřed je pomalejší
- mazání z prostředka je pomalejší
- při vkládání a mazání se musí prvky posouvat

Složitost pole

Operace	Složitost
přístup podle indexu	$O(1)$
změna prvku podle indexu	$O(1)$
vyhledání hodnoty	$O(n)$
vložení doprostřed	$O(n)$
smazání z prostředka	$O(n)$

Deklarace pole

```
int[] ciska;
```

Vytvoření pole

```
ciska = new int[3];
```

Deklarace a vytvoření současně

```
int[] ciska = new int[3];
```

Inicializace hodnotami

```
int[] ciska = { 10, 20, 30 };
```

Procházení pole pomocí `for`

```
int[] cisla = { 10, 20, 30 };

for (int i = 0; i < cisla.Length; i++)
{
    Console.WriteLine(cisla[i]);
}
```

- používá se, když potřebujeme index

Procházení pole pomocí `foreach`

```
int[] cisla = { 10, 20, 30 };

foreach (int cislo in cisla)
{
    Console.WriteLine(cislo);
}
```

- používá se, když nepotřebujeme index

List

- **List** = dynamická kolekce
- v C# se zapisuje jako `List<T>`
- oproti poli se dá jednoduše zvětšovat a zmenšovat
- uvnitř bývá založený na poli
- když se vnitřní pole zaplní, vytvoří se větší pole a prvky se překopírují
- používá se, když nevíme předem přesný počet prvků

Příklad:

```
List<int> cisla = new List<int>();

cisla.Add(10);
cisla.Add(20);
cisla.Add(30);

Console.WriteLine(cisla[0]);
```

Základní operace s `List<T>`

Operace	Význam
Add()	přidá prvek na konec
Remove()	odstraní konkrétní prvek

Operace	Význam
RemoveAt()	odstraní prvek na daném indexu
Contains()	zjistí, jestli prvek existuje
Count	počet prvků
[index]	přístup podle indexu

Příklad:

```
List<string> jmena = new List<string>();

jmena.Add("Jan");
jmena.Add("Eva");

jmena.Remove("Jan");

Console.WriteLine(jmena.Count);
```

Zásobník

Co je zásobník

- **zásobník** = stack
- funguje na principu **LIFO**
- **LIFO** = Last In, First Out
- poslední vložený prvek se odebere jako první
- přidává se i odebírá ze stejného konce

Představa:

```
vrchol zásobníku
[30] ← odebere se první
[20]
[10] ← vloženo jako první
```

Příklad z praxe:

- talíře na sobě
- zpět v historii
- `Ctrl + Z`
- zásobník volání funkcí
- rekurze
- vyhodnocování výrazů

Operace zásobníku

Operace	Význam
Push()	přidá prvek na vrchol
Pop()	odebere a vrátí prvek z vrcholu
Peek()	podívá se na vrchol bez odebrání
Count	počet prvků

Zásobník v Csharp (C#)

```
Stack<int> zasobnik = new Stack<int>();

zasobnik.Push(10);
zasobnik.Push(20);
zasobnik.Push(30);

Console.WriteLine(zasobnik.Pop()); // 30
Console.WriteLine(zasobnik.Peek()); // 20
Console.WriteLine(zasobnik.Count); // 2
```

Procházení zásobníku

```
Stack<string> historie = new Stack<string>();

historie.Push("stranka1");
historie.Push("stranka2");
historie.Push("stranka3");

foreach (string stranka in historie)
{
    Console.WriteLine(stranka);
}
```

- prvky se vypisují od vrcholu zásobníku
- tedy od posledně přidaného

Ruční simulace zásobníku pomocí List<T>

```
List<int> zasobnik = new List<int>();

// push
zasobnik.Add(10);
zasobnik.Add(20);
```

```
zasobnik.Add(30);

// pop
int posledniIndex = zasobnik.Count - 1;
int hodnota = zasobnik[posledniIndex];
zasobnik.RemoveAt(posledniIndex);

Console.WriteLine(hodnota); // 30
```

Fronta

Co je fronta

- **fronta** = queue
- funguje na principu **FIFO**
- **FIFO** = First In, First Out
- první vložený prvek se odebere jako první
- přidává se na konec
- odebírá se ze začátku

Představa:

začátek fronty	konec fronty
[10] → [20] → [30]	
odebírání se zde	přidává se zde

Příklad z praxe:

- fronta u pokladny
- tisková fronta
- zpracování požadavků
- posílání e-mailů na pozadí
- plánování úloh
- fronta zpráv

Operace fronty

Operace	Význam
Enqueue()	přidá prvek na konec fronty
Dequeue()	odebere a vrátí prvek ze začátku fronty
Peek()	podívá se na první prvek bez odebrání
Count	počet prvků

Fronta v Csharp (C#)

```
Queue<string> zpravy = new Queue<string>();

zpravy.Enqueue("email1");
zpravy.Enqueue("email2");
zpravy.Enqueue("email3");

foreach (string zprava in zpravy)
{
    Console.WriteLine(zprava);
}
```

- prvky se vypisují v pořadí od nejstaršího
- tedy v pořadí vložení

Ruční simulace fronty pomocí List<T>

```
List<int> fronta = new List<int>();

// enqueue
fronta.Add(10);
fronta.Add(20);
fronta.Add(30);

// dequeue
int hodnota = fronta[0];
fronta.RemoveAt(0);

Console.WriteLine(hodnota); // 10
```

Poznámka:

- tento způsob je jednoduchý na pochopení
- RemoveAt(0) je ale pomalé
- musí posunout všechny ostatní prvky
- pro skutečnou frontu je lepší použít Queue<T>

Slovník

Co je slovník

- **slovník** = dictionary
- v C# se zapisuje jako Dictionary<TKey, TValue>
- ukládá dvojice:
 - klíč

- hodnota
- místo číselného indexu používá **klíč**
- podle klíče rychle najde odpovídající hodnotu

Příklad:

```
"admin" → "Jan"  
"user" → "Eva"  
"guest" → "Petr"
```

Klíč a hodnota

- **klíč** slouží k vyhledání hodnoty
- klíč musí být v rámci slovníku unikátní
- **hodnota** je uložená data
- hodnoty se mohou opakovat

Příklad:

```
Dictionary<string, int> veky = new Dictionary<string, int>();  
  
veky["Jan"] = 18;  
veky["Eva"] = 19;  
  
Console.WriteLine(veky["Jan"]); // 18
```

Použití slovníku

- konfigurace aplikace
- počítání výskytů
- překlady
- ukládání objektů podle ID
- rychlé vyhledávání podle klíče

Příklady:

```
ID uživatele → uživatel  
název nastavení → hodnota nastavení  
slovo → počet výskytů  
zkratka → celý název
```

Základní operace slovníku

Operace	Význam
Add()	přidá dvojici klíč–hodnota
[key]	přístup k hodnotě podle klíče
ContainsKey()	zjistí, jestli klíč existuje
Remove()	odstraní položku podle klíče
Count	počet položek
Keys	kolekce klíčů
Values	kolekce hodnot

Slovník v Csharp (C#)

```
Dictionary<string, string> konfigurace = new Dictionary<string, string>();

konfigurace.Add("server", "localhost");
konfigurace.Add("database", "skola");

Console.WriteLine(konfigurace["server"]);
```

Bezpečné čtení ze slovníku

```
Dictionary<string, int> veky = new Dictionary<string, int>();

veky["Jan"] = 18;
veky["Eva"] = 19;

if (veky.ContainsKey("Petr"))
{
    Console.WriteLine(veky["Petr"]);
}
else
{
    Console.WriteLine("Klíč neexistuje.");
}
```

TryGetValue

- bezpečnější způsob čtení
- nezpůsobí chybu, pokud klíč neexistuje
- vrátí `true`, pokud klíč existuje
- hodnotu uloží do proměnné

```
Dictionary<string, int> veky = new Dictionary<string, int>();

veky["Jan"] = 18;
veky["Eva"] = 19;

if (veky.TryGetValue("Jan", out int vek))
{
    Console.WriteLine(vek);
}
else
{
    Console.WriteLine("Klíč neexistuje.");
}
```

Složitost slovníku

- slovník je často založený na hashovací tabulce
- klíč se pomocí hashovací funkce převede na index
- díky tomu je vyhledávání velmi rychlé
- průměrná složitost přístupu je $O(1)$

Poznámka:

- v nejhorším případě může být složitost horší
- běžně se ale slovník používá jako rychlá kolekce pro vyhledávání podle klíče

HashSet

- **HashSet** je kolekce unikátních hodnot
- neukládá dvojice klíč–hodnota
- ukládá pouze hodnoty
- každá hodnota se může vyskytovat jen jednou
- používá se, když nás zajímá jen, zda prvek existuje
- často má průměrnou složitost vyhledávání $O(1)$

Příklad:

```
HashSet<string> jmena = new HashSet<string>();

jmena.Add("Jan");
jmena.Add("Eva");
jmena.Add("Jan");

Console.WriteLine(jmena.Count); // 2
Console.WriteLine(jmena.Contains("Eva")); // true
```

Porovnání kolekcí

Kolekce	Princip	Přístup	Typické použití
Pole	pevná velikost, indexy	$O(1)$ podle indexu	pevný počet prvků
List	dynamická velikost, indexy	$O(1)$ podle indexu	proměnný počet prvků
Stack	LIFO	vrchol zásobníku	undo, rekurze
Queue	FIFO	začátek fronty	požadavky, zprávy
Dictionary	klíč–hodnota	$O(1)$ podle klíče	rychlé vyhledávání
HashSet	unikátní hodnoty	$O(1)$ existence prvku	odstranění duplicit

Praktické ukázky

1. Součet prvků pole

```
int[] cisla = { 1, 2, 3, 4, 5 };
int soucet = 0;

for (int i = 0; i < cisla.Length; i++)
{
    soucet += cisla[i];
}

Console.WriteLine(soucet);
```

2. Zásobník: otočení pořadí prvků

```
Stack<string> zasobnik = new Stack<string>();

zasobnik.Push("A");
zasobnik.Push("B");
zasobnik.Push("C");

while (zasobnik.Count > 0)
{
    Console.WriteLine(zasobnik.Pop());
}
```

Výstup:

```
C
B
A
```

3. Fronta: zpracování požadavků

```
Queue<string> fronta = new Queue<string>();

fronta.Enqueue("požadavek 1");
fronta.Enqueue("požadavek 2");
fronta.Enqueue("požadavek 3");

while (fronta.Count > 0)
{
    string pozadavek = fronta.Dequeue();
    Console.WriteLine("Zpracovávám: " + pozadavek);
}
```

4. Slovník: počet výskytů slov

```
string[] slova = { "pes", "kočka", "pes", "pták", "kočka", "pes" };

Dictionary<string, int> pocty = new Dictionary<string, int>();

foreach (string slovo in slova)
{
    if (pocty.ContainsKey(slovo))
    {
        pocty[slovo]++;
    }
    else
    {
        pocty[slovo] = 1;
    }
}

foreach (var dvojice in pocty)
{
    Console.WriteLine(dvojice.Key + ": " + dvojice.Value);
}
```

5. Ruční simulace zásobníku pomocí List<T>

```
List<int> zasobnik = new List<int>();

zasobnik.Add(10); // push
zasobnik.Add(20); // push
zasobnik.Add(30); // push

int posledniIndex = zasobnik.Count - 1;
int hodnota = zasobnik[posledniIndex];
```

```
zasobnik.RemoveAt(posledniIndex); // pop
```

```
Console.WriteLine(hodnota); // 30
```

6. Ruční simulace fronty pomocí `List<T>`

```
List<int> fronta = new List<int>();
```

```
fronta.Add(10); // enqueue
```

```
fronta.Add(20); // enqueue
```

```
fronta.Add(30); // enqueue
```

```
int hodnota = fronta[0];
```

```
fronta.RemoveAt(0); // dequeue
```

```
Console.WriteLine(hodnota); // 10
```